

Java Collection Methods

12 simple programs with sample output

Note: HashSet and Hashtable display order may vary. PriorityQueue display order is not sorted.

Code 1 – ArrayList Methods

```
import java.util.ArrayList;
import java.util.Comparator;

public class ArrayListMethods {
    public static void main(String[] args) {
        ArrayList<String> fruits = new ArrayList<>();

        // 1. add(E e)
        fruits.add("Apple");
        fruits.add("Banana");
        fruits.add("Mango");
        System.out.println("After add(): " + fruits);

        // 2. add(int index, E element)
        fruits.add(1, "Orange");
        System.out.println("After add(index, element): " + fruits);

        // 3. get(int index)
        System.out.println("Element at index 2: " + fruits.get(2));

        // 4. set(int index, E element)
        fruits.set(2, "Grapes");
        System.out.println("After set(): " + fruits);

        // 5. remove(int index)
        fruits.remove(1);
        System.out.println("After remove(index): " + fruits);

        // 6. remove(Object o)
        fruits.remove("Apple");
        System.out.println("After remove(object): " + fruits);

        // 7. contains(Object o)
        System.out.println("Contains Mango? " + fruits.contains("Mango"));

        // 8. size()
        System.out.println("Size of list: " + fruits.size());

        // 9. isEmpty()
        System.out.println("Is list empty? " + fruits.isEmpty());

        // Add duplicate elements for indexOf() and lastIndexOf()
        fruits.add("Mango");
        fruits.add("Apple");
        fruits.add("Mango");

        // 10. indexOf(Object o)
        System.out.println("First index of Mango: " + fruits.indexOf("Mango"));

        // 11. lastIndexOf(Object o)
        System.out.println("Last index of Mango: " + fruits.lastIndexOf("Mango"));

        // 12. sort(Comparator)
        fruits.sort(Comparator.naturalOrder());
        System.out.println("After sorting: " + fruits);

        // 13. clear()
        fruits.clear();
        System.out.println("After clear(): " + fruits);

        // Check whether list is empty
```

```

        System.out.println("Is list empty now? " + fruits.isEmpty());
    }
}

```

Sample Output

```

After add(): [Apple, Banana, Mango]
After add(index, element): [Apple, Orange, Banana, Mango]
Element at index 2: Banana
After set(): [Apple, Orange, Grapes, Mango]
After remove(index): [Apple, Grapes, Mango]
After remove(object): [Grapes, Mango]
Contains Mango? true
Size of list: 2
Is list empty? false
First index of Mango: 1
Last index of Mango: 4
After sorting: [Apple, Grapes, Mango, Mango, Mango]
After clear(): []
Is list empty now? true

```

Code 2 – LinkedList Methods

```

import java.util.LinkedList;

public class LinkedListMethods {
    public static void main(String[] args) {
        LinkedList<String> cities = new LinkedList<>();

        // 1. add(E e)
        cities.add("Hyderabad");
        cities.add("Chennai");
        cities.add("Bangalore");
        System.out.println("After add(): " + cities);

        // 2. addFirst(E e)
        cities.addFirst("Mumbai");
        System.out.println("After addFirst(): " + cities);

        // 3. addLast(E e)
        cities.addLast("Delhi");
        System.out.println("After addLast(): " + cities);

        // 4. get(int index)
        System.out.println("Element at index 2: " + cities.get(2));

        // 5. getFirst()
        System.out.println("First city: " + cities.getFirst());

        // 6. getLast()
        System.out.println("Last city: " + cities.getLast());

        // 7. remove(int index)
        cities.remove(2);
        System.out.println("After remove(index): " + cities);

        // 8. remove(Object o)
        cities.remove("Chennai");
        System.out.println("After remove(object): " + cities);

        // 9. removeFirst()
        String first = cities.removeFirst();
        System.out.println("Removed first city: " + first);
        System.out.println("List after removeFirst(): " + cities);

        // 10. removeLast()
        String last = cities.removeLast();
        System.out.println("Removed last city: " + last);
        System.out.println("List after removeLast(): " + cities);
    }
}

```

```

        // 11. offer(E e)
        cities.offer("Pune");
        System.out.println("After offer(): " + cities);

        cities.offer("Kolkata");
        System.out.println("After another offer(): " + cities);

        // 12. peek()
        System.out.println("Head using peek(): " + cities.peek());
        System.out.println("List after peek(): " + cities);

        // 13. poll()
        String removed = cities.poll();
        System.out.println("Removed using poll(): " + removed);
        System.out.println("List after poll(): " + cities);
    }
}

```

Sample Output

```

After add(): [Hyderabad, Chennai, Bangalore]
After addFirst(): [Mumbai, Hyderabad, Chennai, Bangalore]
After addLast(): [Mumbai, Hyderabad, Chennai, Bangalore, Delhi]
Element at index 2: Chennai
First city: Mumbai
Last city: Delhi
After remove(index): [Mumbai, Hyderabad, Bangalore, Delhi]
After remove(object): [Mumbai, Hyderabad, Bangalore, Delhi]
Removed first city: Mumbai
List after removeFirst(): [Hyderabad, Bangalore, Delhi]
Removed last city: Delhi
List after removeLast(): [Hyderabad, Bangalore]
After offer(): [Hyderabad, Bangalore, Pune]
After another offer(): [Hyderabad, Bangalore, Pune, Kolkata]
Head using peek(): Hyderabad
List after peek(): [Hyderabad, Bangalore, Pune, Kolkata]
Removed using poll(): Hyderabad
List after poll(): [Bangalore, Pune, Kolkata]

```

Code 3 – Vector Methods

```

import java.util.Vector;

public class VectorMethods {
    public static void main(String[] args) {
        Vector<String> subjects = new Vector<>();

        // 1. add(E e)
        subjects.add("Java");
        subjects.add("Python");
        subjects.add("C");
        System.out.println("After add(): " + subjects);

        // 2. addElement(Object obj)
        subjects.addElement("C++");
        System.out.println("After addElement(): " + subjects);

        // 3. get(int index)
        System.out.println("Element at index 1: " + subjects.get(1));

        // 4. set(int index, E element)
        subjects.set(2, "DSA");
        System.out.println("After set(): " + subjects);

        // 5. remove(int index)
        subjects.remove(1);
        System.out.println("After remove(index): " + subjects);
    }
}

```

```

        // 6. removeElement(Object obj)
        subjects.removeElement("C++");
        System.out.println("After removeElement(): " + subjects);

        // 7. size()
        System.out.println("Size of Vector: " + subjects.size());

        // 8. capacity()
        System.out.println("Capacity of Vector: " + subjects.capacity());

        // 9. contains(Object o)
        System.out.println("Contains Java? " + subjects.contains("Java"));
        System.out.println("Contains Python? " + subjects.contains("Python"));
    }
}

```

Sample Output

```

After add(): [Java, Python, C]
After addElement(): [Java, Python, C, C++]
Element at index 1: Python
After set(): [Java, Python, DSA, C++]
After remove(index): [Java, DSA, C++]
After removeElement(): [Java, DSA]
Size of Vector: 2
Capacity of Vector: 10
Contains Java? true
Contains Python? false

```

Code 4 – Stack Methods

```

import java.util.Stack;

public class StackMethods {
    public static void main(String[] args) {
        Stack<String> books = new Stack<>();

        // 1. push(E item)
        books.push("Java");
        books.push("DSA");
        books.push("Python");
        books.push("C++");
        System.out.println("After push(): " + books);

        // 2. peek()
        System.out.println("Top book using peek(): " + books.peek());
        System.out.println("Stack after peek(): " + books);

        // 3. pop()
        String removed = books.pop();
        System.out.println("Removed book using pop(): " + removed);
        System.out.println("Stack after pop(): " + books);

        // 4. empty()
        System.out.println("Is stack empty? " + books.empty());

        // 5. search(Object o)
        System.out.println("Position of DSA: " + books.search("DSA"));
        System.out.println("Position of Java: " + books.search("Java"));
    }
}

```

Sample Output

```

After push(): [Java, DSA, Python, C++]
Top book using peek(): C++
Stack after peek(): [Java, DSA, Python, C++]
Removed book using pop(): C++
Stack after pop(): [Java, DSA, Python]
Is stack empty? false

```

Position of DSA: 2
Position of Java: 3

Code 5 – HashSet Methods

```
import java.util.HashSet;

public class HashSetExample {
    public static void main(String[] args) {
        HashSet<String> animals = new HashSet<>();

        // add()
        animals.add("Lion");
        animals.add("Tiger");
        animals.add("Elephant");
        animals.add("Zebra");
        animals.add("Lion");           // Duplicate is ignored

        System.out.println("Animals: " + animals);

        // contains()
        System.out.println("Contains Tiger? " + animals.contains("Tiger"));
        System.out.println("Contains Horse? " + animals.contains("Horse"));

        // size()
        System.out.println("Number of animals: " + animals.size());

        // remove()
        animals.remove("Zebra");
        System.out.println("After removing Zebra: " + animals);

        // isEmpty()
        System.out.println("Is HashSet empty? " + animals.isEmpty());

        // clear()
        animals.clear();
        System.out.println("After clear: " + animals);

        // Check again
        System.out.println("Is HashSet empty now? " + animals.isEmpty());
    }
}
```

Sample Output

```
Animals: [Zebra, Elephant, Lion, Tiger]
Contains Tiger? true
Contains Horse? false
Number of animals: 4
After removing Zebra: [Elephant, Lion, Tiger]
Is HashSet empty? false
After clear: []
Is HashSet empty now? true
```

Code 6 – LinkedHashSet Methods

```
import java.util.LinkedHashSet;

public class LinkedHashSetMethods {
    public static void main(String[] args) {
        LinkedHashSet<String> languages = new LinkedHashSet<>();

        // 1. add(E e)
        languages.add("Java");
        languages.add("Python");
        languages.add("C");
        languages.add("Java");       // Duplicate will not be added

        System.out.println("After add(): " + languages);
    }
}
```

```

// 2. contains(Object o)
System.out.println("Contains Python? " + languages.contains("Python"));
System.out.println("Contains HTML? " + languages.contains("HTML"));

// 3. size()
System.out.println("Size of LinkedHashSet: " + languages.size());

// 4. remove(Object o)
languages.remove("C");
System.out.println("After remove(): " + languages);

// 5. clear()
languages.clear();
System.out.println("After clear(): " + languages);

// Check size after clearing
System.out.println("Size after clear(): " + languages.size());
}
}

```

Sample Output

```

After add(): [Java, Python, C]
Contains Python? true
Contains HTML? false
Size of LinkedHashSet: 3
After remove(): [Java, Python]
After clear(): []
Size after clear(): 0

```

Code 7 – TreeSet Methods

```

import java.util.TreeSet;

public class TreeSetMethods {
    public static void main(String[] args) {
        TreeSet<Integer> marks = new TreeSet<>();

        // 1. add(E e)
        marks.add(85);
        marks.add(72);
        marks.add(95);
        marks.add(60);
        marks.add(80);
        marks.add(85);    // Duplicate will not be added

        System.out.println("After add(): " + marks);

        // 2. remove(Object o)
        marks.remove(60);
        System.out.println("After remove(): " + marks);

        // 3. contains(Object o)
        System.out.println("Contains 80? " + marks.contains(80));
        System.out.println("Contains 70? " + marks.contains(70));

        // 4. first()
        System.out.println("First element: " + marks.first());

        // 5. last()
        System.out.println("Last element: " + marks.last());

        // 6. higher(E e)
        System.out.println("Element higher than 80: " + marks.higher(80));

        // 7. lower(E e)
        System.out.println("Element lower than 80: " + marks.lower(80));
    }
}

```

```

        // 8. ceiling(E e)
        System.out.println("Element ceiling 82: " + marks.ceiling(82));

        // 9. floor(E e)
        System.out.println("Element floor 82: " + marks.floor(82));

        // 10. pollFirst()
        System.out.println("Removed first: " + marks.pollFirst());
        System.out.println("After pollFirst(): " + marks);

        // 11. pollLast()
        System.out.println("Removed last: " + marks.pollLast());
        System.out.println("After pollLast(): " + marks);
    }
}

```

Sample Output

```

After add(): [60, 72, 80, 85, 95]
After remove(): [72, 80, 85, 95]
Contains 80? true
Contains 70? false
First element: 72
Last element: 95
Element higher than 80: 85
Element lower than 80: 72
Element ceiling 82: 85
Element floor 82: 80
Removed first: 72
After pollFirst(): [80, 85, 95]
Removed last: 95
After pollLast(): [80, 85]

```

Code 8 – PriorityQueue Methods

```

import java.util.PriorityQueue;

public class PriorityQueueMethods {
    public static void main(String[] args) {
        PriorityQueue<Integer> tasks = new PriorityQueue<>();

        // 1. add(E e)
        tasks.add(40);
        tasks.add(10);
        tasks.add(30);

        System.out.println("After add(): " + tasks);

        // 2. offer(E e)
        tasks.offer(20);
        System.out.println("After offer(): " + tasks);

        // 3. peek()
        System.out.println("Head using peek(): " + tasks.peek());
        System.out.println("Queue after peek(): " + tasks);

        // 4. poll()
        System.out.println("Removed using poll(): " + tasks.poll());
        System.out.println("Queue after poll(): " + tasks);

        // 5. remove(Object o)
        tasks.remove(30);
        System.out.println("After remove(30): " + tasks);

        // 6. contains(Object o)
        System.out.println("Contains 20? " + tasks.contains(20));
        System.out.println("Contains 50? " + tasks.contains(50));

        // 7. size()
    }
}

```

```

        System.out.println("Size of PriorityQueue: " + tasks.size());
    }
}

```

Sample Output

```

After add(): [10, 40, 30]
After offer(): [10, 20, 30, 40]
Head using peek(): 10
Queue after peek(): [10, 20, 30, 40]
Removed using poll(): 10
Queue after poll(): [20, 40, 30]
After remove(30): [20, 40]
Contains 20? true
Contains 50? false
Size of PriorityQueue: 2

```

Code 9 – ArrayDeque Methods

```

import java.util.ArrayDeque;

public class ArrayDequeMethods {
    public static void main(String[] args) {
        ArrayDeque<String> customers = new ArrayDeque<>();

        // 1. addFirst(E e)
        customers.addFirst("Rahul");
        customers.addFirst("Arjun");
        System.out.println("After addFirst(): " + customers);

        // 2. addLast(E e)
        customers.addLast("Priya");
        customers.addLast("Sneha");
        System.out.println("After addLast(): " + customers);

        // 3. offerFirst(E e)
        customers.offerFirst("Kiran");
        System.out.println("After offerFirst(): " + customers);

        // 4. offerLast(E e)
        customers.offerLast("Anjali");
        System.out.println("After offerLast(): " + customers);

        // 5. peekFirst()
        System.out.println("First customer: " + customers.peekFirst());

        // 6. peekLast()
        System.out.println("Last customer: " + customers.peekLast());

        // 7. pollFirst()
        System.out.println("Removed first customer: " + customers.pollFirst());
        System.out.println("After pollFirst(): " + customers);

        // 8. pollLast()
        System.out.println("Removed last customer: " + customers.pollLast());
        System.out.println("After pollLast(): " + customers);
    }
}

```

Sample Output

```

After addFirst(): [Arjun, Rahul]
After addLast(): [Arjun, Rahul, Priya, Sneha]
After offerFirst(): [Kiran, Arjun, Rahul, Priya, Sneha]
After offerLast(): [Kiran, Arjun, Rahul, Priya, Sneha, Anjali]
First customer: Kiran
Last customer: Anjali
Removed first customer: Kiran
After pollFirst(): [Arjun, Rahul, Priya, Sneha, Anjali]
Removed last customer: Anjali

```

After pollLast(): [Arjun, Rahul, Priya, Sneha]

Code 10 – HashMap Methods

```
import java.util.HashMap;

public class HashMapMethods {
    public static void main(String[] args) {
        HashMap<Integer, String> students = new HashMap<>();

        // 1. put(K key, V value)
        students.put(101, "Rahul");
        students.put(102, "Priya");
        students.put(103, "Arjun");

        System.out.println("After put(): " + students);

        // 2. get(Object key)
        System.out.println("Student with ID 102: " + students.get(102));

        // 3. remove(Object key)
        students.remove(103);
        System.out.println("After remove(): " + students);

        // 4. containsKey(Object key)
        System.out.println("Contains key 101? " + students.containsKey(101));

        // 5. containsValue(Object value)
        System.out.println("Contains value Priya? "
            + students.containsValue("Priya"));

        // 6. keySet()
        System.out.println("Keys: " + students.keySet());

        // 7. values()
        System.out.println("Values: " + students.values());

        // 8. entrySet()
        System.out.println("Entries: " + students.entrySet());

        // 9. size()
        System.out.println("Size of HashMap: " + students.size());

        // 10. isEmpty()
        System.out.println("Is HashMap empty? " + students.isEmpty());

        // 11. getOrDefault(Object key, V defaultValue)
        System.out.println("Student with ID 101: "
            + students.getOrDefault(101, "Not Found"));

        System.out.println("Student with ID 105: "
            + students.getOrDefault(105, "Not Found"));

        // 12. clear()
        students.clear();
        System.out.println("After clear(): " + students);

        System.out.println("Is HashMap empty now? " + students.isEmpty());
    }
}
```

Sample Output

```
After put(): {101=Rahul, 102=Priya, 103=Arjun}
Student with ID 102: Priya
After remove(): {101=Rahul, 102=Priya}
Contains key 101? true
Contains value Priya? true
Keys: [101, 102]
```

```

Values: [Rahul, Priya]
Entries: [101=Rahul, 102=Priya]
Size of HashMap: 2
Is HashMap empty? false
Student with ID 101: Rahul
Student with ID 105: Not Found
After clear(): {}
Is HashMap empty now? true

```

Code 11 – LinkedHashMap Methods

```

import java.util.LinkedHashMap;

public class LinkedHashMapMethods {
    public static void main(String[] args) {
        LinkedHashMap<Integer, String> products = new LinkedHashMap<>();

        // 1. put(K key, V value)
        products.put(101, "Laptop");
        products.put(102, "Mobile");
        products.put(103, "Headphones");
        products.put(104, "Keyboard");

        System.out.println("After put(): " + products);

        // 2. get(Object key)
        System.out.println("Product with ID 102: " + products.get(102));

        // 3. remove(Object key)
        products.remove(103);
        System.out.println("After remove(): " + products);

        // 4. containsKey(Object key)
        System.out.println("Contains key 101? " + products.containsKey(101));
        System.out.println("Contains key 105? " + products.containsKey(105));

        // 5. keySet()
        System.out.println("Keys: " + products.keySet());

        // 6. values()
        System.out.println("Values: " + products.values());

        // 7. entrySet()
        System.out.println("Entries: " + products.entrySet());
    }
}

```

Sample Output

```

After put(): {101=Laptop, 102=Mobile, 103=Headphones, 104=Keyboard}
Product with ID 102: Mobile
After remove(): {101=Laptop, 102=Mobile, 104=Keyboard}
Contains key 101? true
Contains key 105? false
Keys: [101, 102, 104]
Values: [Laptop, Mobile, Keyboard]
Entries: [101=Laptop, 102=Mobile, 104=Keyboard]

```

Code 12 – Hashtable Methods

```

import java.util.Hashtable;
import java.util.Enumeration;

public class HashtableMethods {
    public static void main(String[] args) {
        Hashtable<Integer, String> countries = new Hashtable<>();

        // 1. put(K key, V value)
        countries.put(1, "India");
    }
}

```

```

countries.put(2, "Japan");
countries.put(3, "Canada");
countries.put(4, "Australia");

System.out.println("After put(): " + countries);

// 2. get(Object key)
System.out.println("Country with key 2: " + countries.get(2));

// 3. remove(Object key)
countries.remove(4);
System.out.println("After remove(): " + countries);

// 4. containsKey(Object key)
System.out.println("Contains key 1? "
    + countries.containsKey(1));

// 5. containsValue(Object value)
System.out.println("Contains value Japan? "
    + countries.containsValue("Japan"));

// 6. keys()
System.out.println("Keys:");
Enumeration<Integer> keys = countries.keys();

while (keys.hasMoreElements()) {
    System.out.println(keys.nextElement());
}

// 7. elements()
System.out.println("Values:");
Enumeration<String> values = countries.elements();

while (values.hasMoreElements()) {
    System.out.println(values.nextElement());
}

// 8. size()
System.out.println("Size of Hashtable: " + countries.size());

// 9. isEmpty()
System.out.println("Is Hashtable empty? " + countries.isEmpty());
}
}

```

Sample Output

```

After put(): {4=Australia, 3=Canada, 2=Japan, 1=India}
Country with key 2: Japan
After remove(): {3=Canada, 2=Japan, 1=India}
Contains key 1? true
Contains value Japan? true
Keys:
3
2
1
Values:
Canada
Japan
India
Size of Hashtable: 3
Is Hashtable empty? false

```